MTH251 2021S1 Exam

Feel free to contact me at maths_and_programming@outlook.sg if you need tuition. I provide university level tuition for the following SUSS units:

* ENG201 Linear Systems Analysis and Design
* ENG301 Microprocessor Programming
* ICT101 Computational Thinking
* ICT133 Structured Programming
* ICT162 Object Oriented Programming
* MTD215 Application of C++ in Multimedia
* ICT231 Data Structures and Algorithms I
* ICT232 Data Structures and Algorithms II
* MTH105 Fundamentals of Mathematics
* MTH107 Calculus I
* MTH108 Calculus II
* MTH316 Multivariable Calculus
* MTH319 Mathematical Methods I
* MTH320 Mathematical Methods II
* MTH207 Linear Algebra
* MTH219 Fundamentals of Statistics and Probability
* MTH220 Statistical Methods and Inference
* MTH355 Basic Mathematical Optimisation

Donaldson Tan

MTH251

Timed Online Assignment – January Semester 2021

Data Structures and Algorithms I

Monday, 17 May 2021 **4:00 pm – 6:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **EIGHT (8)** questions and comprises **EIGHT (8)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, your Full Name, and SUSS PI Number. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAXYZ_RaphaelLee_T1923161 (omit D/O, S/O)*.

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a PDF or Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the PDF or Word file provided it does not exceed the file size limit of 500MB.

7. For answers which cannot be typed and require to be hand-written, ensure that question number is clearly stated on each page. All uploaded hand written answers must be clear, readable and complete. Marks will not be awarded for un-readable or incomplete images.

8. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

9. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

10. Unless otherwise specified, full details of the workings for each question are to be submitted.

Answer all questions. (Total 100 marks)

Question 1

(a) The number of operations in algorithms A and B are $2nlog_2(n)$ and n^2, respectively. Solve n_0 such that A is better than B for $n \geq n_0$. Show the procedures.

(5 marks)

(b) Analyse the time complexity of the following algorithm that computes the cumulative average of a list.

```
def cum_avg (L):
    n = len(L)
    result = [0] * n
    total = 0
    for i in range(n):
        total += L[i]
        result[i] = total / (i+1)
    return result
```

(5 marks)

Question 2

Implement an algorithm in Python to solve the problem of reversing a list, e.g., if the algorithm accepts [2,3,4,1], then it returns [1,4,3,2]. Note that the time complexity of the algorithm should be no worse than $O(n/2)$.

(10 marks)

Question 3

(a) Given a collection of medical records, we assume that each record has multiple fields, including name, age, gender, etc. Among list, tuple and string in Python, which data structure would you choose to store each record? Which data structure would you choose to store the entire collection of records? Discuss why.

(5 marks)

(b) Interpret the following code and write its output, where initialize_nested_list aims to initialize a nested list of zeros, i.e., [[0], [0], …].

```
def initialize_nested_list (n):
        L = [[0]] * n
        return L

list_1 = initialize_nested_list (5)
list_1[0][0] = 1
print (list_1)
```

(5 marks)

(c) Identify if there is any bug in the code in Question 3(b). If yes, correct the code.

(5 marks)

Question 4

(a) Given that my_q is an empty instance of the Queue ADT, the following operations are performed. Interpret the operations and write down the elements in my_q.

```
my_q.enqueue(7);
my_q.enqueue(11);
my_q.dequeue();
my_q.size();
my_q.dequeue();
my_q.enqueue(8);
my_q.enqueue(10);
```

(5 marks)

(b) If the my_q in Question 4(a) is implemented by an array-based list in Python, and the first element of my_q has the memory address of 1234. What is the memory address of the last element in my_q if each memory address occupies one byte? Justify your answer.

(5 marks)

(c) If the my_q in Question 4(a) is implemented by an array-based list in Python, what is the size of the underlying dynamic array if three more enqueue operations are performed? Justify your answer.

(5 marks)

Question 5

(a) Complete the code below to implement an algorithm in Python to remove a node in a doubly linked list called myDLL. The myDLL object has references to the head and tail of the linked list. Note that each node object has a next reference, a prev reference and an element reference.

```
def remove_node (myDLL, node):
        element = node.element
        # complete code here

        return element
```

(6 marks)

(b) Complete the code below to implement an algorithm in Python to remove a node in a singly linked list called mySLL. The mySLL object has references to the head and tail of the linked list. Note that each node object has a next reference and an element reference.

```
def remove_node (mySLL, node):
        element = node.element
        # complete code here

        return element
```

(9 marks)

Question 6

Consider the following Python implementation of the Queue ADT using a singly linked list.

```
class SLLQueue:
    class Node:
        def __init__(self, element, next):
            self.element = element
            self.next = next
    def __init__(self):
        self.head = None
        self.tail = None

    def enqueue(self, e):
        newest = self.Node(e, None)
        if self.head is None:
            self.head = newest
            self.tail = newest
        else:
            newest.next = self.head
            self.head = newest

    def dequeue(self):
        if self.head is None:
            return "Queue is empty"
        if self.head is self.tail:
            answer = self.tail.element
            self.head = self.tail = None
            return answer
        answer = self.tail.element
        node = self.head
        while node.next is not self.tail:
            node = node.next
        node.next = None
        self.tail = node
        return answer
```

(a) Analyse the time complexity of the enqueue and dequeue operations. Explain your analysis.

(5 marks)

(b) Describe the problem of the code that causes computational issues using the enqueue or dequeue operations. No code is required.

(5 marks)

Question 7

Based on the tree shown below in Figure Q7, answer the following questions.

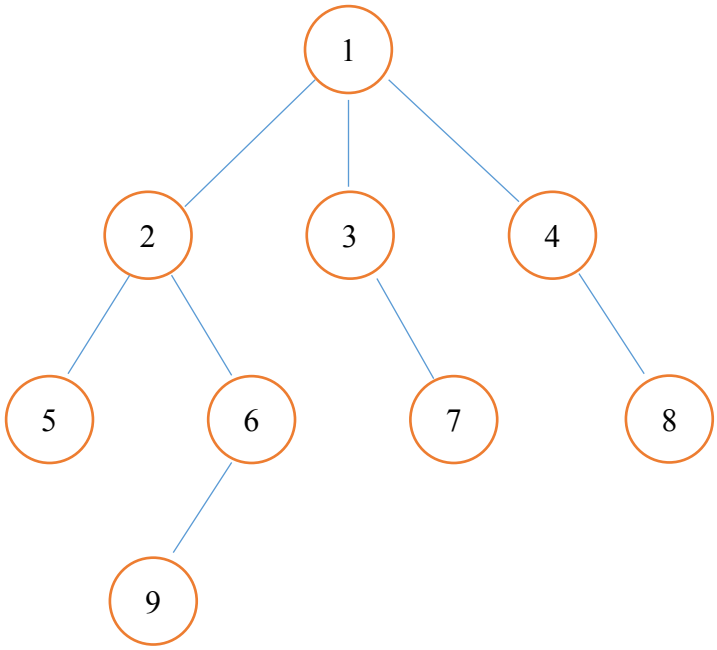


Figure Q7

(a) How many internal nodes in the tree?

(2 marks)

(b) What is the degree of node 6?

(2 marks)

(c) What is the depth of node 7?

(2 marks)

(d) What are the siblings of node 5?

(2 marks)

(e) What is the overall order of visits in preorder traversal? Analyse its time complexity.

(7 marks)

Question 8

(a) Complete the code below to implement an algorithm in Python to remove the first occurrence of a node in an array-based list called myList. Do not use built-in methods in Python list, e.g., myList.remove(node).

```
def remove_node (myList, node):
        # complete code here
        # myList.size denotes the current size of myList
        # removing an element requires decrementing the size by one

        return myList
```

(6 marks)

(b) Analyse the time complexity of the algorithm in Question 8(a). Which data structure(s) is efficient for element removal at arbitrary positions?

(4 marks)

----- END OF PAPER -----

1(a) $T_A = 2n\log_2 n$, $T_B = n^2$

A better than B

$\Leftrightarrow T_A \leq T_B$

$2n\log_2 n \leq n^2$

since $n > 0$

$2\log_2 n \leq n$

$\log_2 n^2 \leq \log_2 2^n$

$n^2 \leq 2^n$

n	n^2	2^n	$n^2 \leq 2^n$
1	1	2	T
2	4	4	T
3	9	8	F
4	16	16	T
5	25	32	T
6	36	64	T

$\therefore n \geq 4 \quad \therefore n_0 = 4$

(b)

```
def cum_avg(L):
    n = len(L)                          # 1 assignment, 1 read = 2 steps
    result = [0] * n                    # 1 assignment, insert n elements = n+1 steps
    total = 0                           # 1 assignment = 1 step
    for i in range(n):                  # n comparison = n steps
        total += L[i]                   # 1 addition, 1 assignment for n-1 loops = 2n-2 steps
        result[i] = total/(i+1)         # 1 addition, 1 division, 1 assignment for n-1 loops = 3n-3 steps
    return result                       # 1 return = 1 step
```

total number of steps $= 2 + (n+1) + 1 + n + (2n-2) + (3n-3) + 1$

$= 5n$

time complexity $= O(n)$

2.

```
def reverse(L: list):
    if len(L) > 0:                      # 1 comparison = 1 step
        i = 0                           # 1 assignment = 1 step
        j = len(L) - 1                  # 1 assignment = 1 step
        while i < j                     # n/2 + 1 comparisons = n/2 + 1 steps
            L[i], L[j] = L[j], L[i]     # 2 assignments = 2 steps
            i = i + 1                   # 1 assignment, 1 addition = 2 steps
            j = j - 1                   # 1 assignment, 1 subtraction = 2 steps
    return L                            # 1 return = 1 step
```

over $\frac{n}{2}$ loops

total number of steps $= 1 + 1 + 1 + (\frac{n}{2}+1) + \frac{n}{2}(2+2+2) + 1$

$= 5 + 9(\frac{n}{2})$

$= O(\frac{n}{2})$

3(a) I would use a tuple for each medical record because:

- a tuple can contain multiple data types for different fields
- a tuple is more memory efficient than a list.

I would use a list to store the entire collection because:

- a list is mutable, making it easy to add or delete a medical record
- each medical record can be looked up by its index number
- a linear search can be performed to find a specific record

(b) [[], [], [], [], []]

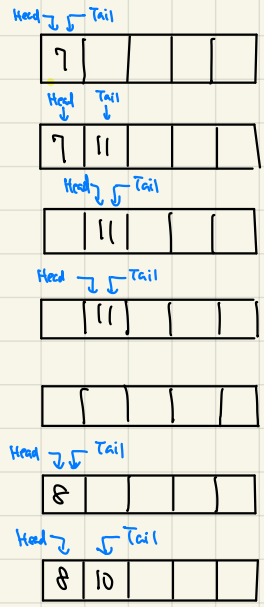
(c) The bug is the implementation of initialize_nested_list().

```
def initialize_nested_list(n: int) -> List[List[int]]:
    return [[0] for _ in range(n)]
```

4(a)

operation	response
my_q.enqueue(7)	True
my_q.enqueue(11)	True
my_q.dequeue()	7
my_q.size()	1
my_q.dequeue()	11
my_q.enqueue(8)	True
my_q.enqueue(10)	True

my_q contents



array-based circular queue implementation

(b) The second element is located at the next byte after the memory address of the first element.

Address of second element = 1234+1 = 1235

(c) Assuming the dynamic array was initialized at size 5, 3 additional enqueue operations only meant the array capacity is consumed entirely but didn't trigger the resize operation. Hence, the size of the underlying dynamic array remains at 5.

5(a)

```
def remove_node(myDLL, node):

    # assume valid input

    element = node.element.

    # complete the code here
    if myDLL.head is None:
        raise Exception("Empty List")

    # assumes the DLL uses a sentinel node
    prev = node.prev
    next = node.next
    prev.next = next.

    # reset the deleted node
    node.prev = None
    node.next = None

    return element
```

(b)

```
def remove_node(mySLL, node):

    # assume valid input
    element = node.element

    # complete the code here

    if mySLL.head == node:
        # remove head
        mySLL.head = node.next
        node.next = None

    else:
        # traverse to target node
        prev = mySLL.head
        current = mySLL.head.next
        while current is not None:
            if current == node:
                prev.next = node.next
                node.next = None
                if node == mySLL.tail
                    mySLL.tail = prev
                break
            prev = current
            current = current.next
    return element
```

6(a)

```
def enqueue(self, e):
        newest = self.Node(e, None)
        if self.head is None:
                self.head = newest
                self.tail = newest
        else:
                newest.next = self.head
                self.head = newest
```

Time complexity for enqueuing an empty queue is the same as enqueuing a non-empty queue

worst case time complexity for enqueue operation
= 1 assignment + 1 node creation + 1 comparison + 2 assignments
= 5 steps
= $O(1)$

```
def dequeue(self):
        if self.head is None:
                return "Queue is empty"
        if self.head is self.tail:
                answer = self.tail.element
                self.head = self.tail = None
                return answer
        answer = self.tail.element
        node = self.head
        while node.next is not self.tail:
                node = node.next
        node.next = None
        self.tail = node
        return answer
```

worst-case time complexity for dequeue operations happens when dequeuing a queue of size > 1

worst case time complexity for dequeue operation
= 2 comparisons + 2 assignments +
(n-1) comparisons + (n-2) assignments +
2 assignments + 1 return
= $2n + 4$ steps
= $O(n)$

6(b) Consider the following steps performed on empty queue myq:

myq.enqueue("Queue is empty")
item = myq.dequeue()

The dequeue operation returns the string "The Queue is empty" which is indistinguishable from the error message "The Queue is empty". This causes computational issue as there is no way to tell if a valid element was removed from the queue. Instead, raise an exception to indicate error.

The 2nd issue with the dequeue operation is that it is $O(n)$ because of updating the tail pointer after the removal of the tail node. If the underlying data structure of the Queue ADT is a double-linked list instead of a single-linked list, then updating the tail pointer would be $O(1)$.

7(a) 5 internal nodes - nodes 1, 2, 3, 4, 6 are internal nodes because they have children.

(b) node 6 has 2 child, so the degree is 1.

(c) node 7 has 2 ancestors, so height = 2

(d) node 2 has 2 children - nodes 5 and 6, thus the siblings of node 5 is node 6

(7) 1, 2, 5, 6, 9, 3, 7, 4, 8

time complexity of each visit is $O(1)$

each node is visited once.

$\therefore$ time complexity of pre-order traversal

$= n \times O(1)$

$= O(n)$

8(a)

```
def remove_node(myList, node):
    i = 0
    while i < myList.size:
        if myList[i] == node:
            j = i+1
            while j < myList.size
                myList[j-1] = myList[j]
                j = j+1
            break
        i = i+1
    myList.size -= 1
    return myList
```

(b) let n be size of myList
let k be the position of the node to be removed

time complexity = 2 assignments + k comparisons + k comparisons + 1 assignment + 1 addition + $(n-k+1)$ comparisons + $(n-k)$ subtractions + $(n-k)$ assignments + $(n-k)$ addition + $(n-k)$ assignment + 1 break + k addition + k assignment + 1 subtraction + 1 assignment + 1 return

$= 5n-k+9$

$= O(5n-k)$

$= O(5n)$ since $0 \le k \le n$

$= O(n)$

A positional list ADT with double-linked list as its underlying implementation would be an efficient data structure for element removal at arbitrary positions as each removal is $O(1)$